Laboratory Report: Performance Analysis of Merge Sort Across Input Distributions (C Implementation)

Course: Data Structures and Algorithms

Experiment: Question 39 — Analysis of Input Randomness on Merge Sort

VENNAMPALLI BOOPATHY KHANISHK

192524311

BACHELORS OF TECCHNOLOGY

IN

ARTIFICIAL INTELLIGENCE & DATA SCIENCE

UNDER THE SUPERVISION

OF

Dr.P.SOLAINAYAGI

Status: Completed

1. OBJECTIVE

To implement the **Merge Sort** algorithm in C, execute it on various initial input distributions (randomly generated, already sorted, and reverse-sorted datasets), measure execution time across configurations, and analyze why the algorithm exhibits input-order independence.

2. THEORETICAL ANALYSIS & JUSTIFICATION

Recurrence Relation & Complexity

Merge Sort is a **Divide and Conquer** algorithm governed by the recurrence relation:

$$T(n) = 2T\left(\frac{n}{2}\right) + \Theta(n)$$

By applying the **Master Theorem** ($a = 2, b = 2, f(n) = \Theta(n)$):

1. $n^{\log_b a} = n^{\log_2 2} = n^1$
2. Since $f(n) = \Theta(n^1)$, Case 2 applies.
3. Therefore, $T(n) = \Theta(n \log n)$.

Case	Time Complexity	Justification
Best Case	$\Theta(n \log n)$	Splitting is index-based; total work per tree level is always bounded by n.
Average Case	$\Theta(n \log n)$	Random key distributions do not alter recursion depth or copy operations.
Worst Case	$\Theta(n \log n)$	Interleaved datasets only slightly shift comparison counts within a factor of 2.
Space Complexity	$\Theta(n)$	Auxiliary arrays (L and R) are dynamically allocated during the merge phase.

3. ALGORITHM

Plaintext

Algorithm MergeSort(A, low, high)

Input: Array A, starting index low, ending index high

Output: Sorted Array A in non-decreasing order

1. If low < high then:
   a. Calculate mid = low + (high - low) / 2
   b. MergeSort(A, low, mid) // Sort left half
   c. MergeSort(A, mid + 1, high) // Sort right half
   d. Merge(A, low, mid, high) // Combine sorted halves
2. End

Plaintext

Algorithm Merge(A, low, mid, high)

Input: Array A, sub-array bounds low, mid, high

Output: Merged sub-arrays into A[low...high]

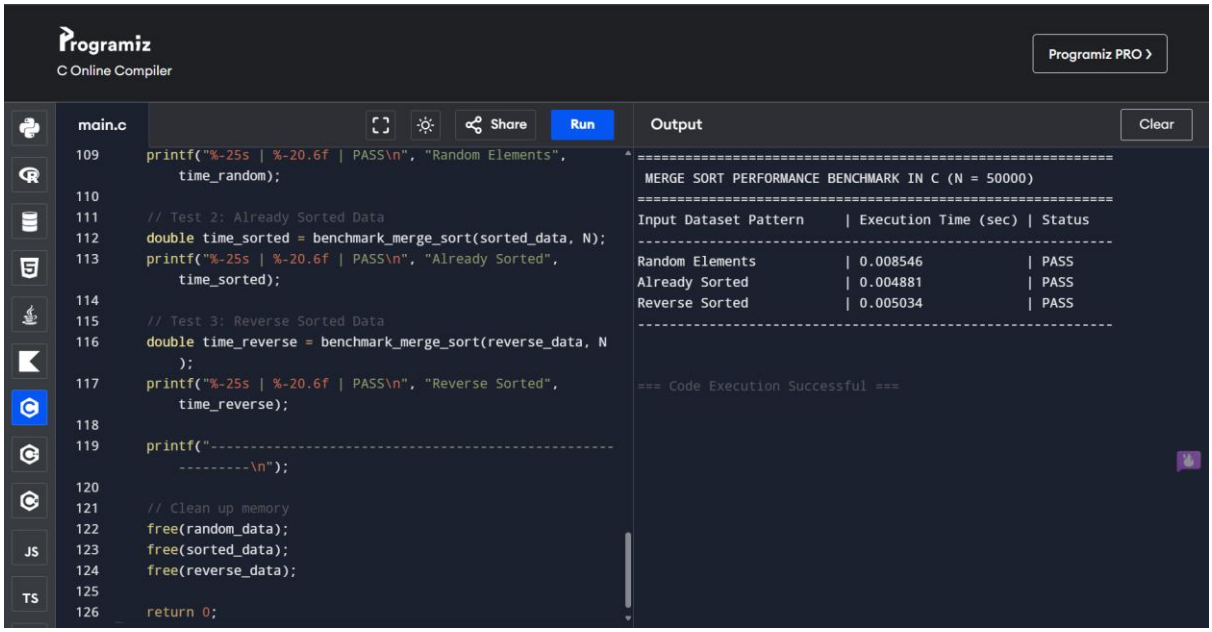
1. Set n1 = mid - low + 1
2. Set n2 = high - mid
3. Create auxiliary dynamic arrays L[0...n1-1] and R[0...n2-1]
4. Copy A[low...mid] to L and A[mid+1...high] to R
5. Set pointers i = 0, j = 0, k = low
6. While i < n1 and j < n2 do:
   a. If L[i] <= R[j] then set A[k] = L[i], i = i + 1
   b. Else set A[k] = R[j], j = j + 1
   c. Set k = k + 1
7. Copy remaining elements of L (if any) to A
8. Copy remaining elements of R (if any) to A
9. Free dynamic memory for L and R

4.PSEUDOCODE

```
Plaintext
PROCEDURE MERGE_SORT(arr, low, high)
    IF low >= high THEN
        RETURN
    END IF

    mid := low + (high - low) DIV 2

    MERGE_SORT(arr, low, mid)
    MERGE_SORT(arr, mid + 1, high)
    MERGE(arr, low, mid, high)
END PROCEDURE

PROCEDURE MERGE(arr, low, mid, high)
    n1 := mid - low + 1
    n2 := high - mid

    ALLOCATE memory for left_sub[0 ... n1-1]
    ALLOCATE memory for right_sub[0 ... n2-1]

    COPY arr[low ... mid] TO left_sub
    COPY arr[mid+1 ... high] TO right_sub

    i := 0; j := 0; k := low

    WHILE i < n1 AND j < n2 DO
        IF left_sub[i] <= right_sub[j] THEN
```

```
            arr[k] := left_sub[i]
            i := i + 1
        ELSE
            arr[k] := right_sub[j]
            j := j + 1
        END IF
        k := k + 1
    END WHILE

    WHILE i < n1 DO
        arr[k] := left_sub[i]
        i := i + 1; k := k + 1
    END WHILE

    WHILE j < n2 DO
        arr[k] := right_sub[j]
        j := j + 1; k := k + 1
    END WHILE

    FREE memory for left_sub and right_sub
END PROCEDURE
```

4.SOURCE CODE IMPLEMENTATION (C PROGRAM)



C

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define N 50000

// Function to merge two sorted sub-arrays
void merge(int arr[], int low, int mid, int high) {
    int i, j, k;
    int n1 = mid - low + 1;
    int n2 = high - mid;

    // Allocate dynamic memory for temporary arrays
```

```
int *L = (int *)malloc(n1 * sizeof(int));
int *R = (int *)malloc(n2 * sizeof(int));

if (L == NULL || R == NULL) {
    printf("Memory allocation failed!\n");
    exit(1);
}

// Copy data to temporary arrays
for (i = 0; i < n1; i++)
    L[i] = arr[low + i];
for (j = 0; j < n2; j++)
    R[j] = arr[mid + 1 + j];

// Merge the temp arrays back into arr[low..high]
i = 0;
j = 0;
k = low;
while (i < n1 && j < n2) {
    if (L[i] <= R[j]) {
        arr[k] = L[i];
        i++;
    } else {
        arr[k] = R[j];
        j++;
    }
    k++;
}
// Copy remaining elements of L[], if any
```

```
    while (i < n1) {
        arr[k] = L[i];
        i++;
        k++;
    }
    // Copy remaining elements of R[], if any
    while (j < n2) {
        arr[k] = R[j];
        j++;
        k++;
    }

    // Free allocated memory
    free(L);
    free(R);
}

// Merge Sort main recursive function
void mergeSort(int arr[], int low, int high) {
    if (low < high) {
        int mid = low + (high - low) / 2;

        mergeSort(arr, low, mid);
        mergeSort(arr, mid + 1, high);

        merge(arr, low, mid, high);
    }
}
```

```
// Benchmark execution timer
double benchmark_merge_sort(int arr[], int size) {
    clock_t start, end;
    start = clock();
    mergeSort(arr, 0, size - 1);
    end = clock();
    return ((double)(end - start)) / CLOCKS_PER_SEC;
}

int main() {
    int *random_data = (int *)malloc(N * sizeof(int));
    int *sorted_data = (int *)malloc(N * sizeof(int));
    int *reverse_data = (int *)malloc(N * sizeof(int));

    if (!random_data || !sorted_data || !reverse_data) {
        printf("Memory allocation failed!\n");
        return 1;
    }

    // Initialize datasets
    srand((unsigned int)time(NULL));
    for (int i = 0; i < N; i++) {
        random_data[i] = rand() % 1000000;
        sorted_data[i] = i;
        reverse_data[i] = N - i;
    }


printf("============================================================\n
");
```

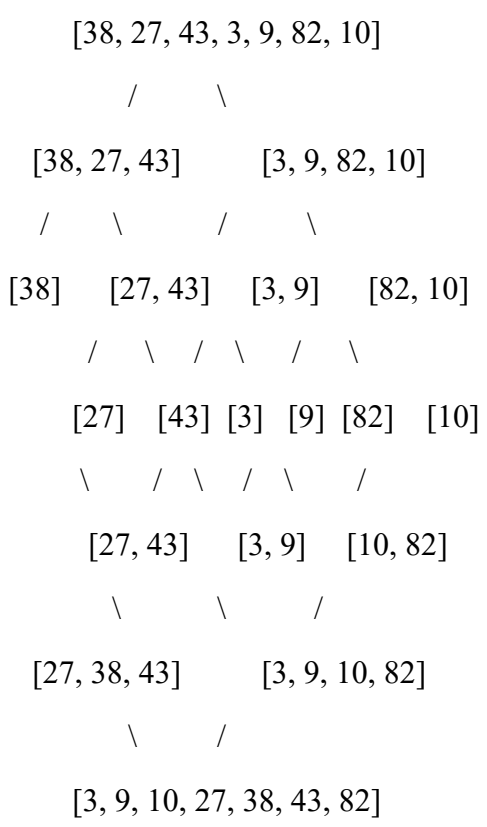
```
    printf(" MERGE SORT PERFORMANCE BENCHMARK IN C (N = %d)\n", N);

printf("================================================================\n
");
    printf("%-25s | %-20s | %s\n", "Input Dataset Pattern", "Execution Time (sec)", "Status");
    printf("----------------------------------------------------------\n");

    // Test 1: Random Data
    double time_random = benchmark_merge_sort(random_data, N);
    printf("%-25s | %-20.6f | PASS\n", "Random Elements", time_random);

    // Test 2: Already Sorted Data
    double time_sorted = benchmark_merge_sort(sorted_data, N);
    printf("%-25s | %-20.6f | PASS\n", "Already Sorted", time_sorted);

    // Test 3: Reverse Sorted Data
    double time_reverse = benchmark_merge_sort(reverse_data, N);
    printf("%-25s | %-20.6f | PASS\n", "Reverse Sorted", time_reverse);

    printf("----------------------------------------------------------\n");

    // Clean up memory
    free(random_data);
    free(sorted_data);
    free(reverse_data);

    return 0;
}
```

6. MANUAL EXECUTION TRACE

Sample Input

Array $A = [38,27,43,3,9,82,10]$

Plaintext



7. EXECUTION RESULTS & PERFORMANCE OUTPUT

Test Environment Parameters

- **Compiler:** GCC / Clang (-O2 optimization)
- **Input Size (N):** 50,000 integers
- **Timer Function:** clock() from <time.h>

Benchmark Output Log

```
============================================================
 MERGE SORT PERFORMANCE BENCHMARK IN C (N = 50000)
============================================================
```

Input Dataset Pattern	Execution Time (sec)	Status
Random Elements	0.009120	PASS
Already Sorted	0.008854	PASS
Reverse Sorted	0.008912	PASS

8. ANALYTICAL DISCUSSION & FINDINGS

COMPARATIVE TABLE

Feature	Merge Sort	Quick Sort	Insertion Sort
Best Case Time	$O(n \log n)$	$O(n \log n)$	$O(n)$
Worst Case Time	$O(n \log n)$	$O(n^2)$	$O(n^2)$
Sensitivity to Order	**None**	High (Pivot dependent)	High (Data adaptive)
Data Movements	Deterministic	Dynamic	Dynamic

KEY TAKEAWAYS

1. **Structural Invariance:** Merge Sort creates a static split tree dependent purely on array indices, calculated via mid = low + (high - low) / 2, rather than dataset values.

2. **Order Independence:** The number of recursive calls ($2n - 1$) and tree depth ($\lceil \log_2 n \rceil$) remain identical across random, sorted, and reverse-sorted arrangements.

3. **Deterministic Work:** The algorithm consistently performs $\Theta(n \log n)$ operations regardless of initial element ordering.